

Exam 1

summary
sheet

Basic Python and Pycharm

INTERACTIVE SHELL

- provides interface to communicate with it
- works like a calculator
- immediate responses

EDITOR PANE

- can write statements in this pane & save them as files
- can ask python to perform entire set of statements one after the other

DEBUGGING

- preprocessing to find typos
- learn to read code & predict what will happen when you run it
- comment along to make debugging easier

PYTHON

CONSOLE / SHELL

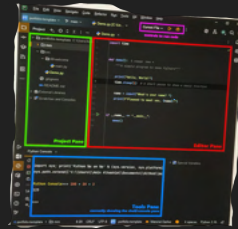
- can try out problems in here but does not save
- can assign values to variables

PYCHARM

→ a kind of writing program to be used instead of IDE

CHARACTERISTICS

- 1) Project Pane - project menu
- 2) Editor - where you create program files
- 3) Python Console - also called SHELL
 - ↳ >>> prompt (means its ready)



ILLUSTRATION

Basic Python Data Types

NUMBERS

- ① Integers ② Floating Point (Real)
- These 2 shall be the ones used in class
- If both operands are integers, then the result is an integer.
- If at least one operand is a floating-point number, then the result is a float too.
- PS: The single slash (/) is an exception: it always returns a float.

There are 3 division-related operators

- ① / : decimal division
↳ gives normal calculator result for dividing 2 numbers
- ② // : quotient division
↳ returns only the integer part, it rounds down to the nearest integer less than the value
- ③ % : remainder operator
↳ gives the remainder of the division of its two operands

Data Types

STRINGS

- Collection of characters
- Strings & characters are written the same in Python
- Strings are written with quotes " " before & after them
- PS: Only tripled-quoted strings, eg " " " can have multiple lines in them

Some functions:

Example	Meaning
<code>len()</code>	The len function returns the number of characters in its argument
<code>"foo" + "bar"</code>	The plus operator concatenates the two strings together
<code>"foo" * 3</code>	The times operator concatenates the string with itself the number of times given
<code>"a" in s</code>	The in operator checks if first string occurs in second string
<code>s[2]</code>	The square bracket operator (also known as access operator) returns the character at the given position, zero-based
<code>s[2:5]</code>	The square bracket operator with a colon (also called slicing) returns a substring starting at 2 and ending before 5

LISTS

- A list is a collection of data
- You write a list by surrounding the data with square brackets
eg [1, 2, 3]
- Both lists & strings share some of the same operations

ILLUSTRATION:

```
int1 = [5, 1, 9, 2]
int2 = ['Nauru', 'Kiribati', 'Vanuatu']
len(int1)
len(int2)
int3 = int1 + int2
int4 = ['a', 'b'] * 5
'Kiribati' in int2
1551 in int2
int1[0]
int2[2]
int1[2:9]
```


ORIGINS

- from programming language Logo
- meant to teach children
- useful for drawing graphics, particularly fractals & are used in multi-agent simulations

IDEA EXPLAINED

- each turtle has a pen tied to its tail
- you tell it where to move & whether to hold its tail up/down
- the tail is up if it's moving without drawing a line
- if it's down it leaves a line behind it

SCREEN

- an object that represents the window, where the turtle will live
↳ it has methods that allow you to change its appearance.
- A Turtle in the screen represents the turtle that lives in the window
↳ it has methods that you use to ask the turtle to move, to raise or lower its pen, and to change its color and other attributes.

Module

- MODULE**
- files that contain tools that are not automatically flooded when you run Python
- MODULE** { parcels / containers }
- objects } these can be run using the "import command"
- these can also be used as a serial message.

* When you import a module python makes a special namespace for that module. You must tell python to look

- in order to access that namespace you must tell Python to look in that namespace by attaching the name of the module followed by a period
 e.g. my_turtle = turtle.Turtle(), window = turtle.Screen()

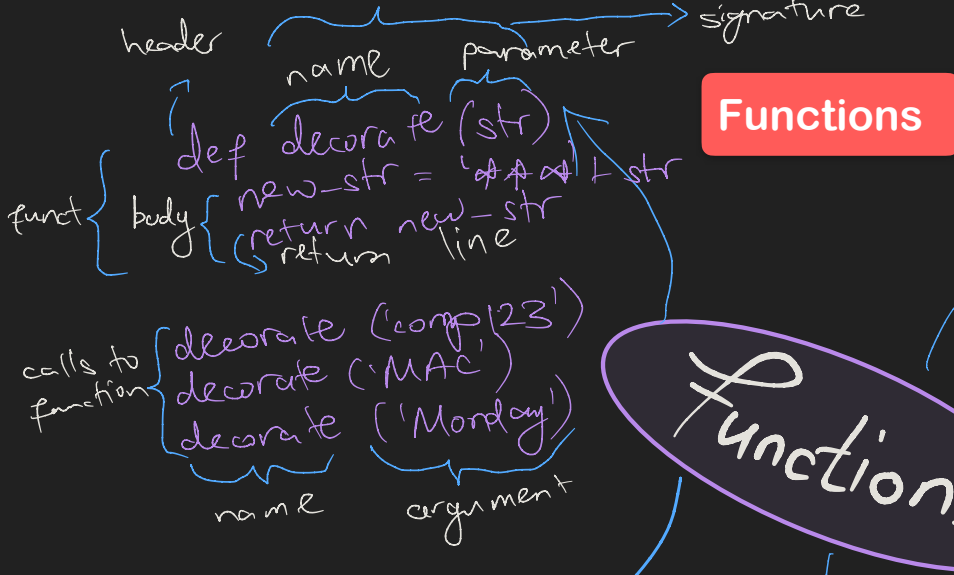
The Turtle Module provides tools for creating a window and drawing a turtle that moves around in that window.
 ↳ To do that we need 2 kind of objects: Screens & Turtles.

OBJECT

- in computer science an object represents a noun
- it usually has data & actions associated with it
- The actions associated with an object are accessed by calling a method, which is like a function that belongs to that particular object.

LOOP

- To avoid repeating typing, and the chance for error to creep in, we use loops to tell the computer to repeat some actions
- It's written as follows
 - for var in sequence:
statements to be repeated (indented)
- The word after for must be a variable name. This special variable is called the loop variable.
- The next must be in, and then we must specify a list of values.



Functions

Definitions

- named sequence of statements that belong together
- can put as many statements as you like
- `def name(parameters):` statements
- has a header line (starts with keyword `def` ends with `:`)
- a body consisting of 1 or more python statements
- parameter can be empty or separated by commas

Parameters

- parameter list is known as formal parameters
- when you use a function, you provide values to the formal parameters
- a function needs certain information to do its work, these values are often called arguments
- actual parameters

Functions

- defining a new function does not make the function run
- for this we need a function call/function invocation

Functions that Return Value

- ① `print(abs(s))` - returns absolute #
- ② `print(math.pow(2, 3))` - takes 2 arguments
base & exponent
and brings highest value
- ③ `print(max)`
- ④ `print(range)`
- ⑤ `print(int)`

Unit Testing

- A Test Case expresses requirements for a program in a way that can be checked automatically.
- A unit test is one form of an automatic procedure eg. a function
- > a collection of unit tests is called a test suite
- Writing notes as you go makes it easier to know what each unit test/test suite was meant to do in the case that it fails to run
- A way to implement unit tests with Python is `assert` eg.


```
assert type(9/5) == int
assert type(9.0//5) == int
```

Returns

- Temporary variables make debugging easier
- temporary variables are examples of local variables
- return statement only returns value of a function not expression

Code Decomposition

→ this is identifying potential functions, creating functions, removing repetitive code, making code more efficient, and debugging code with functions

→ try to make a loop that can be used multiple times so that the code does not become redundant

- The pattern of iterating the updating of a variable is commonly referred to as the accumulator pattern
↳ the variable is referred to as the accumulator

Each of the functions we write can be used and called from other functions we write.

↳ The process of breaking a problem into smaller subproblems is called functional decomposition

When working with functions it is really important to know the order in which statements are executed

this is called the flow of execution

★ Statements are executed one at a time from top to bottom

- function definitions do not alter the flow of execution of the program, but remember that statements inside the function are not executed until the function is called.

code 82:

```
def FUD 1:  
    if ...  
    else ...  
def FUD 2:  
    if ...  
    print ...  
def FUD 3:  
    if ...  
    continue ...
```

Flow of execution diagram: 1 → 2 → 3 → 4 → 5

• When running a function skip the body until that function is called i.e. when reaching code for a function

Function (PRACTICE) 2.0

★ To deal with increasingly complex programs, we are going to suggest a technique called incremental development

★ The ability to functions by using other functions is called composition.

→ Why should we write a loop instead of repeating cat?
• If you write the exact same statements in more than one place, then you have to debug each place separately.
→ If you loop then you only debug it once.

At the beginning of your page import all modules needed

6.11. A Turtle Bar Chart

Recall from our discussion of modules that there were a number of things that turtles can do. Here are a couple more tricks (remember that they are all described in the module documentation).

- We can get a turtle to display text on the canvas at the turtle's current position. The method is called `write`. For example, `alex.write("Hello")` would write the string `hello` at the current position.
- One can fill a shape (circle, semicircle, triangle, etc.) with a fill color. It is a two-step process. First you call the method `begin_fill`, for example `alex.begin_fill()`. Then you draw the shape. Finally, you call `end_fill`, for example `alex.end_fill()`. Then you draw the shape.
- We've previously set the color of our turtle - we can now also set its fill color, which need not be the same as the turtle and the pen color. To do this, we use a method called `fillcolor`, for example, `alex.fillcolor("red")`.

Ok, so can we get tess to draw a bar chart? Let us start with some data to be charted.

```
xs = [48, 117, 208, 249, 168, 268, 228]
```

Corresponding to each data measurement, we'll draw a simple rectangle of that height, with a fixed width. Here is a simplified version of what we would like to create.



We can quickly see that drawing a bar will be similar to drawing a rectangle or a square. Since we will need to do it a number of times, it makes sense to create a function, `drawBar`, that will need a turtle and the height of the bar. We will assume that the width of the bar will be 40 units. Once we have the function, we can use a basic for loop to process the list of data values.

6.12. Glossary

chatterbox function

A function which interacts with the user (using `input` or `print`) when it should not. Silent functions that just convert their input arguments into their output results are usually the most useful ones.

composition (of functions)

Calling one function from within the body of another, or using the return value of one function as an argument to the call of another.

dead code

Part of a program that can never be executed, often because it appears after a `return` statement.

fruitful function

A function that yields a return value instead of `None`.

incremental development

A program development plan intended to simplify debugging by adding and testing only a small amount of code at a time.

None

A special Python value. One use in Python is that it is returned by functions that do not execute a return statement with a return argument.

return value

The value provided as the result of a function call.

scaffolding

Code that is used during program development to assist with development and debugging. The unit test code that we added in this chapter are examples of scaffolding.

temporary variable

A variable used to store an intermediate value in a complex calculation.

```
def drawBar(t, height):
    """ Get turtle t to draw one bar, of height. """
    t.left(90)           # Point up the left side
    t.forward(height)    # Draw up the left side
    t.right(90)          # width of bar, along the top
    t.forward(40)        # and down again!
    t.right(90)          # Put the turtle facing the way we found it.
    t.forward(height)    # assume xs and tess are ready
    t.left(90)

...
for v in xs:
    drawBar(tess, v)
```

It is a nice start! The important thing here was the mental chunking. To solve the problem we first broke it into smaller pieces. In particular, our chunk is to draw one bar. We then implemented that chunk with a function. Then, for the whole chart, we repeatedly called our function.

Next, at the top of each bar, we'll print the value of the data. We will do this in the body of `drawBar` by adding `t.write(str(height))` as the new fourth line of the body. Note that we had to turn the number into a string. Finally, we'll add the two methods needed to fit each bar.

The one remaining problem is related the fact that our turtle lives in a world where position (0,0) is at the center of the drawing canvas. In this problem, it would help if (0,0) were in the lower left hand corner. To solve this we can use our `setWindowCoordinates` method to rescale the window. While we are at it, we should make the window will need to be proportional to the number of bars (the number of data values) where each has a width of 40. Using this information, we can compute the coordinate system that makes sense for the data set. To make it look nice, we'll add a 10 unit border around the bars.

Here is the complete program. Try it and then change the data to see that it can adapt to the new values. Note also that we have stored the data values in a list and used a few list functions. We will have much more to say about lists in a later chapter.